What is the difference between a compiler and an interpreter?

A compiler generally translates code into another representation before execution, while an interpreter executes or evaluates program instructions using a runtime. Real languages can combine compilation and interpretation, so avoid treating this as a strict language-by-language division. In preparing for Accenture interview questions for freshers, this distinction is useful.

What is an API?

An application programming interface defines how software components communicate. A RESTful HTTP API may expose resources through endp This example strengthens Accenture interview questions for freshers preparation.oints such as GET /products and POST /orders, returning structured data and status codes.

What is OOP encapsulation?

Enca Check this principle against Accenture interview questions for freshers examples.psulation combines state and methods and controls access to internal values. For example, an Account class can keep balance private and expose a validated deposit method.

What is the difference between SQL WHERE and HAVING?

WHERE filters rows before grouping, while HAVING filters aggregate groups after GROUP BY. For example, HAVING COUNT(*) > 5 finds Use this scenario when practising Accenture interview questions for freshers. departments with more than five employees.

How would you debug a slow web page?

Measure fir A clear explanation improves Accenture interview questions for freshers preparation.st: network waterfall, browser performance profile, image sizes and render-blocking scripts. Then prioritize improvements such as caching, compressed images and reducing unnecessary JavaScript. Validate results rather than guessing.

What is a primary key?

A primary key identifies each row uniquely and cannot be null. It supports relational integrity and often has an index, depending on the database.

Explain the difference between call by value and call by reference.

“Call by value and call by reference are two different ways to pass arguments to functions, and understanding the distinction is important for writing efficient code. In call by value, the function receives a copy of the variable’s value, so any changes made inside the function don’t affect the original variable. This is safer because you

can’t accidentally modify data, but it uses more memory since you’re copying the data.

In call by reference, the function receives the memory address of the variable, so changes inside the function do affect the original variable. This is more memory-efficient and useful when you need to modify the original data or when you’re working with large objects, but you have to be careful about unintended side effects. For example, languages like Java use call by value for primitives like integers and effectively call by reference for objects, which is why modifying an object inside a method affects the original object.”

What are the advantages of using recursion?

“Recursion offers several practical advantages for solving certain types of problems. First, it dramatically simplifies code for problems that have a natural recursive structure, like traversing trees, calculating factorials, or implementing divide-and-conquer algorithms. The code tends to be more readable and elegant compared to iterative solutions with complex loop structures.

Second, recursion naturally breaks down complex problems into smaller, similar subproblems, which makes the solution easier to understand, debug, and maintain. For example, when traversing a tree structure, the recursive approach mirrors the tree’s natural hierarchy.However, I’m also aware of the tradeoffs. Recursion uses more memory because each recursive call adds to the call stack, and poorly implemented recursive functions without proper base cases can cause stack overflow errors. That’s why I always consider whether an iterative solution might be more efficient before choosing recursion, especially for problems with deep recursion depths or when working with limited memory.”

What’s the difference between normalization and denormalization in database design?

“Normalization and denormalization are two different approaches to organizing database tables, and choosing between them depends on your priorities.

Normalization is the process of organizing database tables to reduce redundancy and improve data integrity. You break large tables into smaller, related tables and use keys to establish relationships between them. This prevents data anomalies and saves storage space. For example, instead of storing customer information repeatedly for each order, you'd have a separate customers table and reference it from the orders table using a customer ID. This ensures that when a customer updates their address, you only need to change it in one place.

Denormalization is deliberately introducing redundancy to improve query performance. You might combine tables or duplicate data to avoid costly joins that slow down queries. This speeds up read operations significantly, which is important for analytics workloads, but it makes updates more complex because you have to maintain data consistency across multiple locations.

In practice, I choose normalization for transactional databases where data integrity is critical and you're constantly inserting and updating records. I choose denormalization for analytics databases or data warehouses where query speed matters more than write efficiency and the data is primarily read-only. Many modern systems actually use a hybrid approach, keeping transactional systems normalized but creating denormalized views or data marts for reporting and analytics."

Why there are no global variables in Java?

Answer:-

Global variables are globally accessible. Java does not support globally accessible variables due to following reasons:1)The global variables breaks the referential transparency

2)Global variables creates collisions in namespace

What is method overloading and method overriding?

Answer:-

Method overloading: When a method in a class having the same method name with different arguments is said to be method overloading.

Method overriding: When a method in a class having the same method name with same arguments is said to be method overriding.

What is the difference between a constructor and a method?

Answer:-

A constructor is a member function of a class that is used to create objects of that class. It has the same name as the class itself, has no return type, and is invoked using the new operator.

A method is an ordinary member function of a class. It has its own name, a return type (which may be void), and is invoked using the dot operator.

What is meant by Inheritance and what are its

advantages?

Answer:-

Inheritance is the process of inheriting all the features from a class.

The advantages of inheritance are:-

- reusability of code
- accessibility of variables and methods of the super class by subclasses.

What is SQL, and why is it important?

Answer:-

SQL stands for Structured Query Language, and is the most important data processing language in use today. It is not a complete programming language like Java or C#, but a data sublanguage used for creating and processing database data and metadata. All DBMS products today use SQL.

What is recursion in C?

Answer:-

Using recursion, one function can be called multiple times. This reduces the code length, reducing the overall complexity of the program.

Q1. What is a non-static member function? How is it called?

Answer:

A non-static member function is a function that belongs to an object of a class. It can access both non-static and static data members of the class.

It is called using an object of the class with the dot (.) operator.

Example:

```
#include <iostream>

using namespace std;



class Student {
```

```
public:
    void display() {
        cout << "Hello Student";
    }
};

int main() {
    Student s;
    s.display();
    return 0;
}
```

Q2. Explain different types of semaphores along with their uses.

Answer:

A semaphore is a synchronization mechanism used in operating systems to control access to shared resources by multiple processes or threads.

There are two main types of semaphores:

1. **Binary Semaphore:** It has values 0 or 1. It is used to control access to a shared resource and coordinate processes or threads.
2. **Counting Semaphore:** It can have a non-negative integer value. It is used to manage access to multiple instances of a resource, such as database connections or printer resources.

Q3. How does thrashing cause the performance of a computer to degrade?

Answer:

Thrashing occurs when the operating system spends more time swapping pages between RAM and disk than executing actual processes.

It happens when there is insufficient physical memory for the active processes.

Effects of thrashing:

- CPU utilization decreases.
- Page faults increase.
- System response time becomes slower.
- Overall system performance degrades.

Q4. What is the difference between turnaround time and response time?

Answer:

- **Turnaround Time:** The total time taken from the submission of a process until its completion.
- **Response Time:** The time between the submission of a process and the first response from the system.

Formula:

Turnaround Time = Completion Time − Arrival Time

Response Time = First Response Time − Arrival Time

Q5. A structure and a class use similar syntax. True or False?

Answer:

True.

In C++, structures and classes use similar syntax and can both contain data members and member functions.

The main difference is that structure members are public by default, whereas class members are private by default.

Q6. Write a structure specification that includes 4 float variables called length, breadth, height, and volume. Call this structure box.

Answer:

```
struct box {

    float length;
```

```
    float breadth;

    float height;

    float volume;

};
```

Q7. What technology have you used in your 6th and 8th semester project?

Answer:

In my 6th-semester project, I used technologies such as Python, HTML, CSS, JavaScript, and a database, depending on the project requirements.

In my 8th-semester project, I used Python and machine learning libraries to develop the system, along with tools for data processing and evaluation.

I selected these technologies based on the project requirements and used them to implement the main features.

Note: Replace these technologies with the actual technologies you used in your projects.

Q8. Define the following terms: DBMS, query, and SQL.

Answer:

1. **DBMS (Database Management System):** A software system used to create, store, organize, retrieve, and manage data in a database. Examples include MySQL, Oracle, and PostgreSQL.

2. **Query:** A request to retrieve or manipulate data in a database.
3. **SQL (Structured Query Language):** A standard language used to create, retrieve, update, and delete data in relational databases.

Q9. What is the difference between a primary key and a secondary key in DBMS?

Answer:

- **Primary Key:** A column or combination of columns that uniquely identifies each record in a table. It cannot contain duplicate or NULL values.
- **Secondary Key:** A column or combination of columns used to retrieve or search records based on particular values. It does not necessarily have to be unique.

Example: In a Student table, `Student_ID` can be the primary key, while `Department` can be used as a secondary search key.

Q10. What is XML, and what are some of its applications?

Answer:

XML stands for Extensible Markup Language. It is a markup language used to store, organize, and exchange structured data in a human-readable format.

Applications of XML:

- Data exchange between different applications.
- Configuration files.
- Web services and APIs.
- Document storage.
- Data representation in business systems.

Q11. What is procedural programming? Give some examples.

Answer:

Procedural programming is a programming approach in which a program is divided into procedures or functions. It follows a step-by-step approach to solve a problem.

Examples of procedural programming languages:

- C
- Pascal
- FORTRAN
- BASIC

In procedural programming, the main focus is on functions and the sequence of instructions used to perform a task.